UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

(SECP38843) SPECIAL TOPIC IN DATA ENGINEERING

SECTION 02

GROUP 3

AI ALGORITHM TUTORIAL: CNN

LECTURER: DR. ARYATI BINTI BAKRI

STUDENT NAME	MATRIC NO
GUI KAH SIN	A23CS0080
SABRINA HENG WEI QI	A23CS0265
WOO CHENG SHUAN	A23CS0283

Table of Contents

1.0 Introduction

1.1 What is Image Classification?

Image classification is a computer vision approach that allows a computer system to automatically classify an image in a set of predefined classes. The model is trained on a set of images to understand patterns and features and then it makes a prediction about the category of the image based on its training.

For this project, the image classification was performed with the CIFAR-10 dataset from TenforFlow/Keras. The data has various categories of various coloured images, such as airplane, car, bird, cat, dog, horse, ship and truck. Recognize these categories by analyzing image patterns, colours and shapes, as known as train model.

From the tutorial, the images were preprocessed by normalizing the pixel values and subsequently, trained with deep learning models. The tutorial also compared ANN and CNN approach. The CNN approach was found to be better in recognizing the features of the image. This project gave me a hands-on experience of image classification and how deep learning can be applied to enable visual recognition.

1.2 What is CNN?

A Convolutional Neural Network (CNN) is a type of Deep Learning algorithm that is specifically designed for image processing and computer vision applications. CNN is popular due to its ability to automatically extract key image features like edges, texture and shapes, without human intervention, which are essential for learning about the shape of objects.

CNN has been employed for classification of images from CIFAR-10 dataset in this tutorial. In contrast to traditional Artificial Neural Network (ANN), CNN retains the spatial relationship of the pixel in the image, thus enhancing the classification accuracy. The model goes through various layers, such as convolutional layers, pooling layers, flatten layers and dense layers.

The convolutional layer takes our important features from the image and the pooling layer brings down the size of the image so that the model is more efficient. Then flatten and dense layers classify the last time. The CNN model of this project performed better than ANN because it was able to learn image patterns to a greater extent.

The enhanced CNN model also incorporated dropout, batch normalization and data augmentation methods to enhance model stability and minimize overfitting. The model was able to perform this enhancement, which led to the model generalizing well for unseen images.

1.3 Evaluation of CNN

The CNN model was tested using its training accuracy, validation accuracy and accuracy in prediction of test images. Evaluation is crucial as it quantifies the model's ability to accurately classify images and whether the model can generalize to new data.

The CNN model was able to achieve much higher accuracy than the ANN model in the tutorial. The ANN model was found to be less accurate due to the direct flattening of the image that resulted in the loss of important image information. By contrast, CNN could retain structures in images and extract the meaningful featured, resulting in more accurate predictions.

The model trained well with the accuracy increasing over time as more and more epochs completed. The improved CNN model also exhibited improved validation performance with the use of additional techniques like dropout layers, batch normalization and data augmentation. These techniques minimized over-fitting and increased the model stability.

The model was further tested by predicting its sample images from the testing dataset and then comparing the predicted labels with the actual label. The majority of the predictions were accurate, indicating that the CNN model was able to effectively capture the key elements of the images it was trained on. Overall, the evaluation demonstrated that CNN is more effective and reliable for image classification tasks compared to the basic ANN approach.

2.0 Hands-on Code on Python

In this section, the process of implementing the image classification using Python programming language and Convolutional Neural Network (CNN) is explained. The implementation has been done in the Jupyter Notebook with the help of TensorFlow and Keras libraries. The image classification model was trained and tested with the CIFAR-10 dataset during the entire tutorial.

2.1 Installing TensorFlow Library

The first step in the implementation process was to install the TensorFlow library:

```
[1]: !pip install tensorflow
```

Figure 2.1: Code for install TensorFlow

TensorFlow is an open-source deep learning framework that offers tools and functions for creating AI and neural network models. TensorFlow makes it easy to implement deep learning architectures like Artificial Neural Network (ANN) and Convolution Neural Network (CNN).

This installation step is crucial as the CNN architecture, loading the dataset, training the CNN and evaluating the results rely on TensorFlow functions. The neural network operations in this tutorial cannot be performed without installing TensorFlow.

2.2 Import Libraries

Sevaral libraries were imported before starting the implementation process:

```
[2]: import tensorflow as tf
     from tensorflow.keras import datasets, layers, models
     import numpy as np
     import matplotlib.pyplot as plt
```

Figure 2.2: Code for import libraries

Every library is used for a particular purpose in the development of the model.

Table 1: Library explanation

Library	Explanation
TensorFlow	Used to build and train deep learning models. It provides the computational framework required for neural network operations.
Keras	Imported from TensorFlow as it provides a simpler and more user-friendly interface for building neural network architectures. It allows the model to be constructed layer by layer using Sequential model.
NumPy	Numerical operations and array manipulations were performed using NumPy. Since images are in multidimensional numerical matrices, Numpy is a key tool for efficiently handle image data.
Matplotlib	Used for image visualization. It allows the visualization of training images, prediction and graphical outputs.

2.3 Load CIFAR-10 Dataset

The following command was used to load the CIFAR-10 dataset:

```
[3]: (X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
     Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
     170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 16s 0us/step
```

Figure 2.3: Loading CIFAR_10 dataset

The CIFAR-10 dataset is well-known benchmark dataset commonly used in computer vision and deep learning research. It has 60,000 colour images categorized into 10 different object classes.

The dataset contains 50,000 training images and 10,000 testing images. There are 10 images categories such as airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. Each picture having a size of 32 x 32 x 3 (width x height x RGB color channel) The training dataset was used to learn the model and the testing dataset was used to test the model on unseen data.

2.4 Examining Dataset Shape

The following were used for examining the dimensions of the dataset:

```
[4]: X_test.shape

[4]: (10000, 32, 32, 3)

[5]: X_train.shape

[5]: (50000, 32, 32, 3)

[6]: y_train.shape

[6]: (50000, 1)

[7]: y_test.shape

[7]: (10000, 1)
```

Figure 2.4: Check the shape for training and testing datasets

Based on the output given, it confirms that the dataset was loaded successfully. The output indicates that there are 10,000 testing images and 50,000 training images with 32 x 32 pixels and 3 color channels. Understanding the dimensions of the dataset is important as CNN models need to be trained with the proper input shape.

2.5 Reshaping Label Data

The labels dataset was reshaped with the following code:

```
y_train[:5]

array([[6],
       [9],
       [9],
       [4],
       [1]], dtype=uint8)
```

Figure 2.5.1: Show current label data

```
[9]: y_train = y_train.reshape(-1,)
     y_train[:5]

[9]: array([6, 9, 9, 4, 1], dtype=uint8)

[10]: y_test = y_test.reshape(-1,)
```

Figure 2.5.2: Reshape label data for both training and testing data

In Figure 2.5.1 shows that current label data is in two-dimensional format. Based on Figure 2.5.2, the code use reshape(-1,) to automatically change to the appropraite size of the array, therefore, the array has been reshaped to one-dimensional arrays. This transformation is needed because using 1D array able to facilitate the processing of the labels during classification and evaluation.

2.6 Defining Class Labels

A list of the names of the image categories was created:

```
[11]: classes = ['airplane','automobile','bird','cat','deer','dog','frog','horse','ship','truck']
```

Figure 2.6: List of names of image categories

The labels in the CIFAR-10 dataset are encoded numerically. For example, 0 represent 'airplane' and 9 represent 'truck'. This class list is created for coveting the numerical output to readable object names, thus enhancing the interpretation and visualization of the results.

2.7 Visualizing Sample Images

```
[12]: def plot_sample(X, y, index):
          plt.figure(figsize = (15,2))
          plt.imshow(X[index])
          plt.xlabel(classes[y[index]])
```

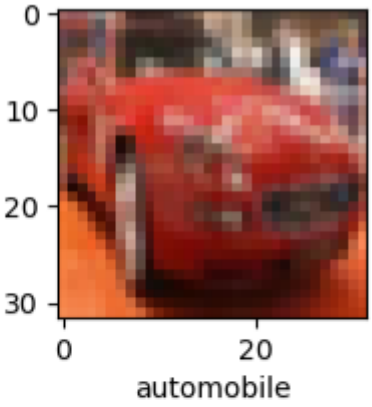
Figure 2.7.1: Function created to show sample images

A custom function named plot_sample (X, y, index) was created to display sample images. The function performs several tasks:

Table 2: Function description

Function	Explanation
plt.figure()	Creates a plotting canvas for image visualization
plt.imshow()	Displays the selected image in the dataset
plt.xlabel()	Displays the corresponding image label below the image

```
[13]: plot_sample(X_train, y_train, 5)
```




```
[14]: plot_sample(X_train, y_train, 501)
```

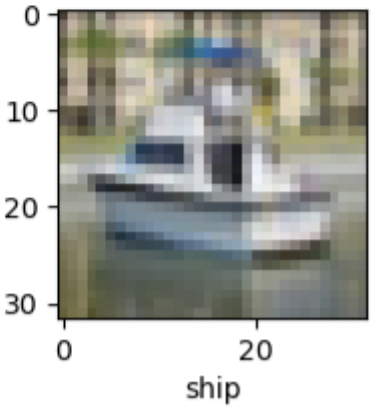


Figure 2.7.2: Output images by using the image visualization function

Figure 2.7.2 shows the output when calling the custom function. This visualization step is needed as it allows to check if the images and labels are loaded properly before the model is trained.

2.8 Data Normalization

```
[15]: X_train = X_train / 255.0
      X_test = X_test / 255.0
```

Figure 2.8: Normalized pixel values of the image

The original pixel values in the dataset were between 0 and 255. These values were normalized to a smaller range of 0 to 1. This preprocessing step is important because large numbers can

make learning more difficult for the neural network and cause the gradients to be unstable during optimization.

Normalization will accelerate the convergence speed, stabilize the training of models and boost the overall performance of deep learning models. It also guarantees that all pixel values are on the same numerical scale.

2.9 Artificial Neural Network (ANN) Model Development

Before implementing the CNN architecture, an Artificial Neural Network model was developed first to compare the performance with CNN.

```
[16]: ann = models.Sequential([
          layers.Flatten(input_shape = (32,32,3)),
          layers.Dense(3000, activation = 'relu'),
          layers.Dense(1000, activation = 'relu'),
          layers.Dense(10, activation = 'softmax')
      ])
```

Figure 2.9: ANN Model Development

In Figure 2.9 shows the code of implementing ANN Model. The flatten layer concert multidimensional image into one-dimensional vector to process by dense layers. Once flattened, the image size became one vector which having 3072 elements.

Next, the dense layers were used as fully connected neural network layers that learned feature relationships from the input data. The first dense layer had 3000 neurons and the second one contains 1000 neurons. Otherwise, ReLU activation function was used in these two layers to improve the learning capability, while the final layer use softmax activation to generate probability values for the ten object categories.

2.10 Compiling and Training the ANN Model

```
ann.compile(optimizer = 'SGD',
            loss = 'sparse_categorical_crossentropy',
metrics = ['accuracy'])

ann.fit(X_train, y_train, epochs = 5)
```

Figure 2.10: Compile and Train ANN Model

The SGD (Stochastic Gradient Descent) optimizer was used to update model weights during training. It was decided to use the sparse categorical crossentropy loss function because the project was a multi-class classification with integer labels.

The training process enabled the ANN model to learn the image patterns from the training set for 5 epochs. The model iteratively updated its internal parameters to reduce prediction errors and enhance classification performance throughout each epoch. The model continuously updated its internal parameters to minimize prediction errors and improve classification accuracy throughout each epoch.

2.11 Predicting ANN Results & Classification Report

```
[17]: from sklearn.metrics import confusion_matrix, classification_report
      import numpy as np
      y_pred = ann.predict(X_test)
      y_pred_classes = [np.argmax(element) for element in y_pred]

      print('classification report: \n', classification_report(y_test, y_pred_classes))
```

Figure 2.11.1: Predict ANN Result and generate report

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
classification report:
              precision    recall  f1-score   support

           0       0.55      0.53      0.54      1000
           1       0.51      0.74      0.60      1000
           2       0.29      0.51      0.37      1000
           3       0.32      0.52      0.40      1000
           4       0.34      0.50      0.41      1000
           5       0.52      0.20      0.29      1000
           6       0.59      0.43      0.50      1000
           7       0.64      0.44      0.52      1000
           8       0.78      0.40      0.53      1000
           9       0.71      0.33      0.45      1000

    accuracy                           0.46     10000
   macro avg       0.53      0.46      0.46     10000
weighted avg       0.53      0.46      0.46     10000
```

Figure 2.11.2: Classification Report

The classification report showed ket evaluation metrics such as precision, recall, F1-score and accuracy. These metrics ave detailed information about the performance of the ANN model and helped to assess the effectiveness of image classification. Based on diagram, the overall accuracy achieved by ANN model was approximately 46%. This means the model correctly classified 46% for the testing images. Although the ANN model could learn certain image patterns, the classification accuracy remained relatively low for image recognition tasks. This result demonstrates that ANN has limitations when handling image datasets because ANN does not preserve spatial relationships between image pixels.

2.12 Visualizing ANN Prediction Result Using Heatmap

```
[18]: import seaborn as sns

[19]: plt.figure(figsize = (14,7))
      sns.heatmap(y_pred, annot = True)
      plt.ylabel('Truth')
      plt.xlabel('Prediction')
      plt.title('Confusion matrix')
      plt.show
```

Figure 2.12.1: Heatmap Visualization Code

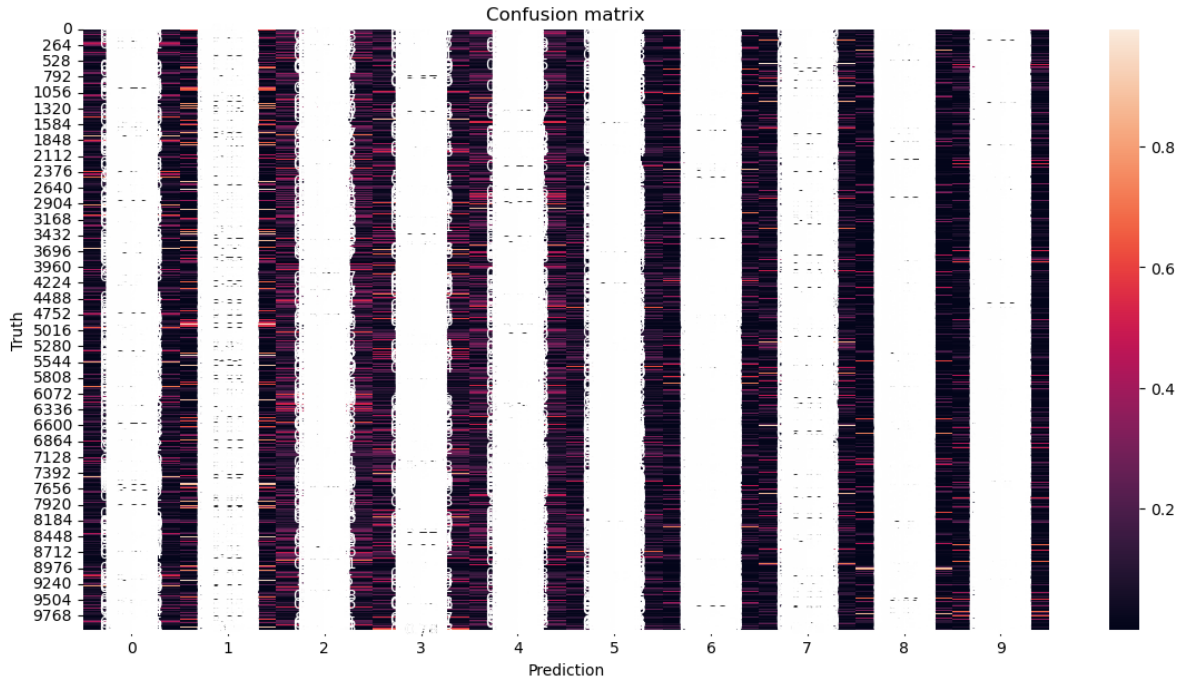


Figure 2.12.2: Heatmap Visualization

After generating the ANN prediction results, a heatmap visualization was created using the Seaborn library to observe the prediction output distribution produced by the model. The heatmap visualization provided insight into the prediction distribution generated by the ANN model. Brighter regions within the heatmap indicated higher prediction probability values, while darker regions represented lower prediction confidence.

2.13 Developing CNN Model

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 2.13: Implement CNN Model

The convolution layers extracted image features such as edges, textures and shapes automatically. The number of filters in the first convolutional layer is 32 and the second layer is 64, suggesting that the first convolutional layer was used to detect visual patterns and the second layer used to extract deeper image features. Next, the max pooling layers were used to shrink the image size, but they kept the vital information from the image. Pooling increased computational efficiency and reduced the risk of overfitting.

Before classification, the extracted feature maps were flattened and converted to one-dimensional vectors. Finally, the dense layers were used to do the final classification task and obtain the probability values for the 10 image categories.

2.14 Compiling and Training CNN Model

```
cnn.compile(optimizer='adam',
            loss = 'sparse_categorical_crossentropy',
            metrics = ['accuracy'])
```

Figure 2.14.1: Compile CNN Model

Adam optimizer was chosen because it achieves faster convergence and more stable learning performance than traditional optimizers.

```
[51]: cnn.fit(X_train, y_train, epochs=10)

Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.4598 - loss: 1.5056
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.5991 - loss: 1.1427
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.6474 - loss: 1.0116
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.6750 - loss: 0.9317
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.6953 - loss: 0.8709
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.7153 - loss: 0.8149
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.7331 - loss: 0.7710
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.7474 - loss: 0.7254
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.7634 - loss: 0.6805
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.7732 - loss: 0.6439
```

Figure 2.14.2: Train CNN Model

The CNN model was trained on the training dataset with the help of the fit() function. The number of epochs (10) means that the model was trained 10 times on the full training set.

In the training process, the model continuously adjusted its internal weights to enhance its prediction ability and minimize classification errors. Two important metrics were generated during training, namely accuracy and loss.

The CNN model has been trained and the results of the training showed that the CNN model improved continuously during the training process. In the first epoch, the model's accuracy was around 45.98% with a loss value of 1.5056. The accuracy slowly improved as the training went on and the loss value kept on decreasing.

At the end of the last epoch, the CNN model reached an accuracy of about 77.32% and loss was reduced to 0.6439. This shows that the CNN model was able to learn the image patterns and enhance its classification ability as times passed.

The CNN model was found to be much more effective than the ANN model used earlier, due to its ability to retain the spatial structure of image pixels and to automatically extract features from the image. The overall training results show that the CNN architecture is effective in learning image features and enhancing the classification accuracy of the CIFAR-10 dataset.

2.15 Evaluating CNN Model

The trained CNN model was evaluated using:

```
cnn.evaluate(X_test, y_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.6852 - loss: 0.9663
[0.9663141369819641, 0.6851999759674072]
```

Figure 2.15: Evaluation CNN Model

The performance of the trained CNN model was evaluated by the unseen testing images with the help of the function evaluate(). The result shows two values, the first value is the loss value while the second value is the accuracy of the model.

Based on the result, the CNN model able to classify around 68.52% of the images correctly in the test set, which means the model did a good job of classifying the testing images. The loss value of 0.9663 represents the error in the prediction during the evaluation process. The CNN model outperformed the ANN model that was implemented earlier. This improvement is due to CNN's ability to maintain spatial relationships between image pixels and the ability to automatically extract features.

2.16 Generating CNN Prediction and Comparing Results

```
[53]: y_pred = cnn.predict(X_test)
      y_pred[:5]
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

```
[53]: array([[2.76146317e-03, 4.27569212e-05, 5.81622962e-03, 6.50383532e-01,
              1.99251785e-03, 2.00311869e-01, 1.06976680e-01, 2.09401493e-04,
              3.12353708e-02, 2.70146498e-04],
             [2.72617072e-01, 1.86480209e-01, 2.47013804e-06, 2.28908048e-09,
              6.04040451e-07, 2.98666675e-10, 5.35737621e-10, 7.44982884e-08,
              5.38855493e-01, 2.04412593e-03],
             [4.27254252e-02, 8.92264366e-01, 5.90548210e-04, 1.33383859e-04,
              4.80723189e-04, 2.07859375e-05, 5.63213762e-05, 2.07946112e-04,
              2.47527342e-02, 3.87678072e-02],
             [4.98157889e-01, 5.93277998e-03, 1.78960781e-03, 6.91091873e-06,
              4.19030926e-04, 5.07568778e-08, 5.67431880e-06, 2.83964573e-06,
              4.93323326e-01, 3.61879502e-04],
             [1.63089760e-06, 1.02475942e-05, 6.58465326e-02, 3.53158005e-02,
              2.63268739e-01, 8.67699180e-03, 6.26755476e-01, 2.00074846e-05,
              1.02818725e-04, 1.79342783e-06]], dtype=float32)
```

```
[54]: y_classes = [np.argmax(element) for element in y_pred]
      y_classes[:5]
```

```
[54]: [np.int64(3), np.int64(8), np.int64(1), np.int64(0), np.int64(6)]
```

```
[55]: y_test[:5]
```

```
[55]: array([3, 8, 8, 0, 6], dtype=uint8)
```

Figure 2.16: Code for generate prediction and compare result

Prediction probability values for each testing image were obtained using the `predict()` function. The CNN model did not output a single class label, but instead it outputted the probabilities for each of the 10 image categories. The values were the confidence of the model for each class. The maximum probability value in this prediction output is `6.50383532e-01` or about `0.6503`. This means that the model was confident of 65% about the image it predicted for a specific category. These probability values do not directly tell you the final predicted class, however. For this reason, the index with the maximum probability value was obtained using the `np.argmax()` function.

Based on the output results, four out of the first five images were classified correctly by the CNN model. Only the third image was misclassified, where the model predicted Class 1 instead of Class 8. The result shows that the CNN model can learn the important image features and have relatively good classification accuracy on the test images that it has not seen before. The prediction comparison also revealed that CNN could make accurate classifications for most of the images in the CIFAR-10 dataset.

2.17 Displaying Prediction Results

The testing images were displayed using the following code:

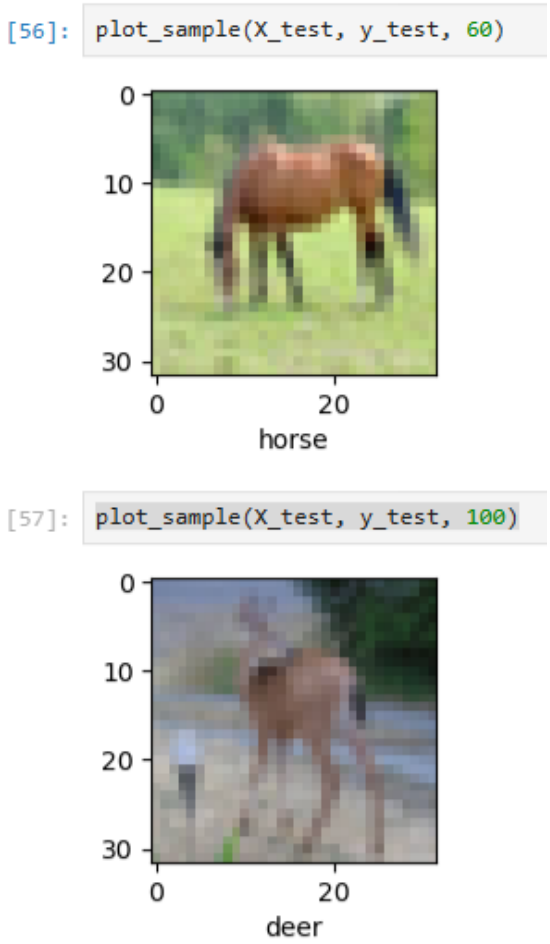


Figure 2.17.1: Image Testing

The predicted category for a selected image was displayed using the following code:

```
[58]: classes[y_classes[60]]

[58]: 'horse'
```

Figure 2.17.2: Predict Category

Displaying prediction results visually is important because it allows direct comparison between the actual image and the predicted category generated by the CNN model. This helps evaluate whether the model can classify images accurately based on visual observation.

Overall, the evaluation results demonstrated that the CNN model achieved considerably better classification performance compared to the ANN model. The CNN architecture successfully extracted important visual features and improved image classification accuracy on the CIFAR-10 dataset.

3.0 Weakness of the current CNN Model

Although the Convolutional Neural Network (CNN) model performed significantly better than the Artificial Neural Network (ANN) model, but there are several drawbacks in the current architecture that impacted the overall classification performance and learning ability of the model.

The first problem with the existing CNN is that the network architecture is shallow where the model implemented has only just two convolutional layers. These layers can be used to extract simple image features like edges, textures and simple shapes. However, the architecture is still shallow but needed for complex image classification tasks. In deep learning, the deeper the CNN architecture, the better it can learn the hierarchical features of images step by step. The early convolutional layers tend to capture low-level features, and the deeper layers capture more complex object structures and semantic representations. For current model, the feature extraction ability is still limited as it has only two convolutional layers, especially when it comes to distinguishing between similar categories of objects in the CIFAR-10 dataset, like cats and dogs.

The second drawback is the lack of regularization methods, such as dropout layers. There is no dropout mechanism to prevent overfitting during the training process in the current CNN model. Overfitting happens when the model learns the patterns of the training data rather than learning the patterns of images. The consequence of this is that the model can perform well during training but perform less well when it is used for classification of new images that it has not seen before. If there are no dropout layers, all neurons are activated throughout the training process, which makes the model more reliant on certain learned features and decreases its ability to generalize.

The third is that no data augmentation is performed during the training of the model. The CNN model was trained with the original images of CIFAR-10 without any image transformation like rotation, horizontal flipping, zooming or shifting. Data augmentation is crucial because it adds diversity to the data set and enables the model to learn various forms of the same object. The model might not perform well on images that are rotated, scaled, or shifted. As a result, the model's generalization capability to real-world variations in images is reduced.

In general, the CNN model presented in this paper proved to be effective in showing the basic implementation of image classification, but the model still had a few limitations in feature extraction, preventing overfitting, and generalizing datasets.

4.0 Enhancement of the current CNN Model

The original CNN model achieved an accuracy of 67.84% on the CIFAR-10 dataset. Although the model was able to classify images reasonably well, there are also several limitations that can be improved to enhance its performance. Three enhancement techniques were applied, which are deeper CNN architecture with more convolutional blocks, dropout regularization and data augmentation.

4.1 Enhancement 1: Deeper Architecture

The original CNN consisted of only two convolutional blocks, which limited its ability to extract complex image features. Therefore, to address this weakness, additional convolutional layers and third convolutional block were added to distinguish similar object more effectively. The enhanced architecture in new_cnn included two convolutional layers in the first and second blocks followed by a third convolutional block with 128 filters.

This deeper architecture allows the model to learn hierarchical image features more effectively, starting from simple edges and textures to more complex object shapes and patterns.

```
deep_cnn = models.Sequential([

     # Input layer
    layers.Input(shape=(32,32,3)),
    # Block 1
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'), layers.MaxPooling2D((2,2)),

    # Block 2
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),

    # Block 3
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 4.1: Deeper Architecture

4.2 Enhancement 2: Dropout Regularization

The original CNN did not contain any mechanism to prevent overfitting. Thus, dropout layers were introduced after pooling layers and before the output layer. Dropout randomly disables a portion of neurons during training to prevent the model from becoming overly dependent on specific neurons and overfitting. This improves the model's ability to generalize to unseen data.

```
new_cnn = models.Sequential([

    layers.Input(shape=(32,32,3)),

    # BLOCK 1
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),

    # NEW: Dropout added to reduceoverfitting prob
    layers.Dropout(0.25),

    # BLOCK 2
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.Conv2D(64, (3,3), activation='relu',  padding='same'),
    layers.MaxPooling2D((2,2)),

    # NEW: Dropout added
    layers.Dropout(0.25),

    # BLOCK 3
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),

    # NEW: Dropout added
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),

    # NEW: Dropout added
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
```

Figure 4.2: Dropout Regularization

4.3 Enhancement 3: Data Augmentation

Data augmentation was implemented to increase the diversity of training images without collecting additional data. Various image transformations such as rotation, shifting, zooming and horizontal flipping were applied during training. This technique helps the model become more robust to image variations and improves its ability to classify unseen images.

```python
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Enhancement 3: Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=15,       # Rotate image
    width_shift_range=0.1,   # Horizontal shift
    height_shift_range=0.1,  # Vertical shift
    horizontal_flip=True,    # Flip image
    zoom_range=0.1           # Zoom in/out
)
```

Figure 4.3: Data Augmentation

5.0 Evaluation Based on CNN and New CNN Model

To evaluate the effectiveness of the proposed enhancements, the performance of the original CNN and the enhanced new_cnn were compared using two commonly used evaluation metrics, accuracy and loss. Accuracy measures the percentage of correctly classified images, while loss indicates the error between the actual labels and predicted outputs. Thus, a higher accuracy and lower loss show a better model performance.

Table 3: Comparison between Original CNN and New CNN

Original CNN	new_cnn

`# Max_pooling - reduces image dimension while preserving features`
`# Returns [loss, accuracy]`
`# loss → how wrong the predictions are on average (lower is better)`
`# accuracy → percentage of images correctly classified`
`cnn.evaluate(X_test, y_test)`
`313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.6784 - loss: 0.9516`
`[0.9515547752380371, 0.6783999800682068]`
	`new_loss, new_accuracy = new_cnn.evaluate(`
	`  X_test,`
	`  y_test,`
	`  verbose=0`
	`)`
	`print("===== New_CNN Evaluation =====")`
	`print(f"Accuracy: {new_accuracy*100:.2f}%")`
	`print(f"Loss: {new_loss:.4f}")`
	`===== New_CNN Evaluation =====`
	`Accuracy: 71.91%`
	`Loss: 0.7985`

5.1 Analysis of Accuracy

The results show that the enhanced new_cnn achieved an accuracy from originally 67.84% to 71.91%. This represents an improvement of 4.07% and indicates the new_cnn was able to classify a larger number of CIFAR-10 images correctly.

The improvement can be attributed to the deeper CNN architecture, which enabled the model to learn more complex and hierarchical image features. Additional convolutional layers allowed the network to capture richer and complex patterns such as textures, shapes and object structures, leading to better classification performance.

Furthermore, the implementation of dropout regularization reduced the risk of overfitting by preventing the model from relying excessively on specific neurons during training. This improved the model's ability to generalize to unseen testing data.

Data augmentation also contributed to the performance improvement by introducing variations in the training images through rotation, shifting, zooming and horizontal flipping. As a result, the model became more robust to changes in object orientation, position and scale.

5.2 Analysis of Loss

The analysis of loss also being tested and the new_cnn also achieved a lower loss value of 0.7985 compared to 0.9516 for the original CNN. The reduction of 0.1531 in loss indicates that the new_cnn has made more accurate predictions and produced the outputs were closer to the actual class labels. A lower loss value suggests that the model learned the underlying patterns of the dataset more effectively and was able to minimize prediction errors during classification. This demonstrates that the enhancements not only increased the number of correct predictions but also improved the overall quality and confidence of the predictions.

5.3 Training and Validation Accuracy

[61]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('New_CNN Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```

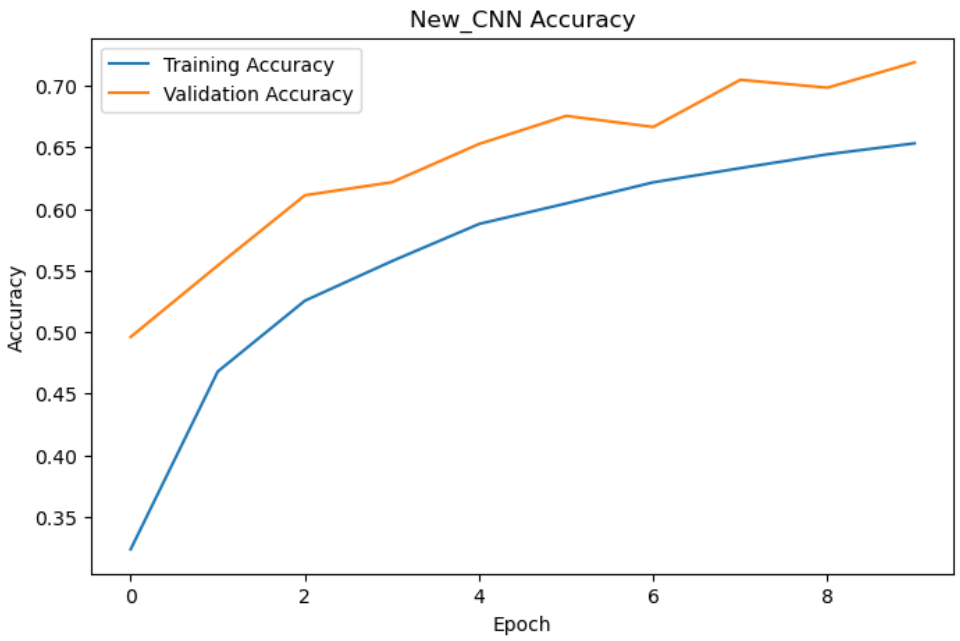


Figure 5.3: Training and Validation Accuracy Graph

Figure 5.3 illustrates the training and validation accuracy throughout the training process. Both curves show a consistent upward trend that indicates the model is successfully learned useful image features over successive epochs. However, the validation accuracy increased steadily and reached approximately 71.91% which is higher than training accuracy. The relatively small gap between two models suggests that the new_cnn has better generalization to unseen data.

5.4 Training and Validation Loss

```
[63]: plt.figure(figsize=(8,5))

      plt.plot(history.history['loss'], label='Training Loss')

      plt.plot(history.history['val_loss'], label='Validation Loss')

      plt.title('New_CNN Loss')
      plt.xlabel('Epoch')
      plt.ylabel('Loss')
      plt.legend()

      plt.show()
```

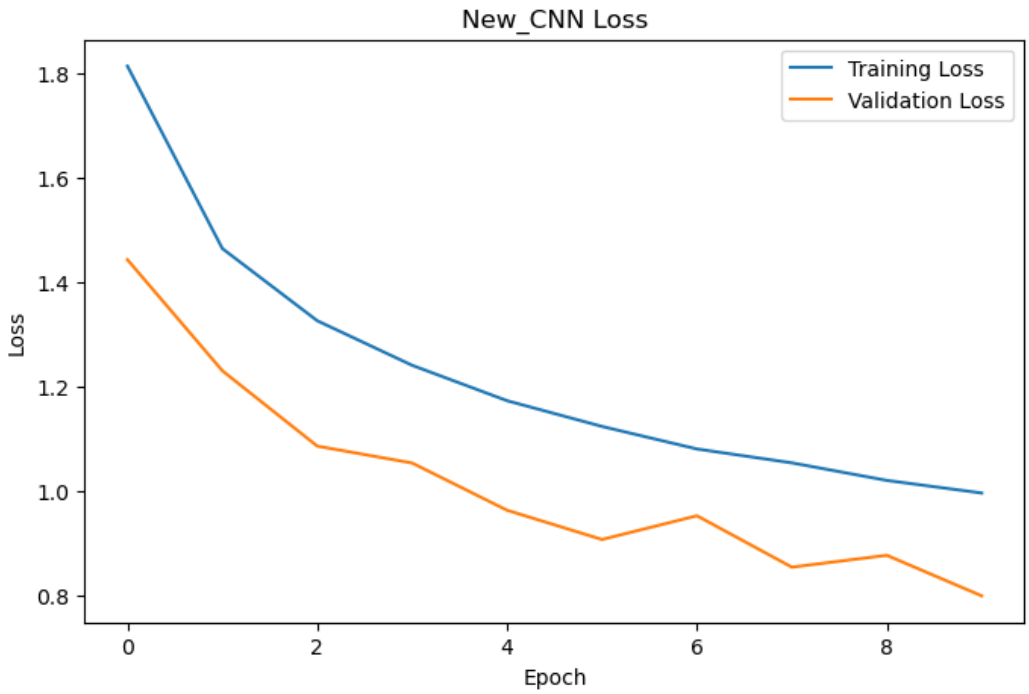


Figure 5.4: Training and Validation Accuracy Graph

Figure 5.4 presents the training and validation loss curves during the model training. The decreasing trend of both curves indicates that the model progressively reduced prediction errors as training developed. The validation loss decreased to 0.7985, demonstrating that the enhanced new_cnn model was able to maintain good performance and reduce error when classifying unseen testing data. The validation loss curve shows no significant fluctuations, indicating that the model is stably learning patterns and the optimization process is effective.

5.5 Confusion Matrix Analysis

```
[44]: import numpy as np
from sklearn.metrics import confusion_matrix
import seaborn as sns

y_pred = new_cnn.predict(X_test)

y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix( y_test, y_pred_classes)

plt.figure(figsize=(8,6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title('New_CNN Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step
```

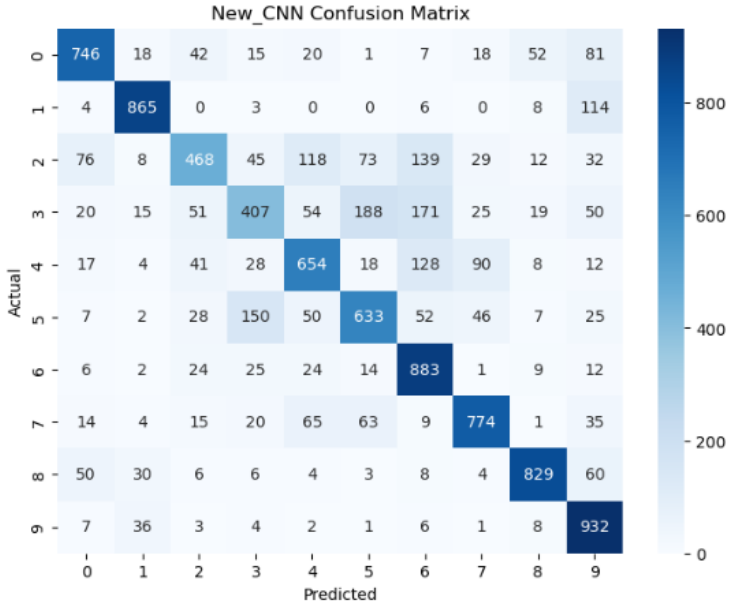


Figure 5.5: new_cnn Confusion Matrix

The confusion matrix reveals that the new_cnn performs best on classes with visually distinct features. Truck achieved 932 correct predictions out of 1000 followed by frog reached 883, automobile scores 865 and ship gets 829. These classes have unique shapes, colours or backgrounds that make the classification easier even at a low resolution.

The most challenging classes are cat, who only scored 407 out of 1000 and bird, who scored 468 out of 1000. The most frequent misclassification in the entire matrix is predicting cat as dog, which happened 188 times out of 1000 cat images. This is because cats and dogs share very similar visual features at 32x32 resolution with similar fur texture and body proportions. Similarly, 150 dog images were also misclassified as cat, confirming this is a two-way confusion problem. Furthermore, bird also struggled because small birds at 32x32 pixels share visual similarities with other animals like deer and frog. As a result, the diagonal values are consistently high across most classes, ensuring that the new_cnn successfully learned to classify the majority of CIFAR-10 images correctly.

Overall, the result of new_cnn demonstrate that it is outperformed than the original CNN model. The accuracy increased from 67.84% to 71.91% while the loss decreased from 0.9516 to 0.7985. These findings ensure that the proposed enhancements successfully improved the CNN model's feature extraction capability, reduced overfitting and increased the ability to generalize to unseen image. Hence, the new_cnn can be considered a more effective image classification model for the CIFAR-10 dataset compared to the original CNN.

6.0 Results and Discussion

6.1 Results

This tutorial trained and compared three models which are ANN, the original CNN and the enhanced CNN (new_cnn). Among these three models, ANN achieved the lowest classification performance as it can only process the images as one-dimensional data and is unable to effectively capture the spatial features. CNN models were better as it is able to extract image features more effectively through convolution and pooling operations.

The experimental results show that the enhanced new_cnn model outperformed the original CNN model on the CIFAR-10 dataset. The original CNN achieved an accuracy of 67.84% with a loss value of 0.9516, while the New_CNN achieved an accuracy of 71.91% with a loss value of 0.7985.

The accuracy increased by 4.07 percentage points, indicating that the enhanced model was able to classify more images correctly. In addition, the lower loss value demonstrates that the new_cnn produced more accurate predictions and reduced classification errors.

The training and validation accuracy curves showed a consistent improvement throughout the training process, while the loss curves gradually decreased, indicating stable learning behaviour.

6.2 Discussion

The improvement in performance can be attributed to the three enhancement techniques implemented in the new_cnn model. First, the deeper CNN architecture enabled the model to learn more complex and meaningful image features. Besides, dropout regularization reduced overfitting by preventing the model from relying too heavily on specific neurons during training. Last but not least, data augmentation increased the diversity of training images, helping the model become more adapt to variations in image orientation, position and scale.

Overall, the enhancements successfully improved the model's classification performance and ability to handle unseen data. However, the achieved accuracy of 71.91% indicates that there is still room for further improvement. Future work may involve increasing the number of training epochs, applying batch normalization or exploring more advanced CNN architectures.

Reflection

Gui Kah Sin

This tutorial provided valuable practical experience in implementing image classification using both Artificial Neural Networks (ANN) and Convolutional Neural Networks (CNN). The step-by-step guidance from downloading the CIFAR-10 dataset, preprocessing and splitting the data into training and testing sets, converting image data into one-dimensional format for ANN and subsequently implementing CNN for improved classification performance, contributed to a better understanding of the complete image classification workflow. The tutorial also introduced the use of Matplotlib for visualizing model performance through accuracy and loss graphs, allowing the learning progress of the models to be monitored effectively. Furthermore, the enhancement of a deeper CNN architecture, dropout regularization and data augmentation demonstrated how model optimization techniques can improve classification accuracy and reduce prediction errors. The results highlighted the flexibility of CNN models, as their performance can be enhanced through the addition of convolutional layers, convolutional blocks and increased training epochs. Overall, this tutorial emphasized the importance of selecting appropriate model architectures and optimization techniques for image classification tasks.

Sabrina Heng Wei Qi

This tutorial clarified how deep learning techniques can be used for image classification using Convolutional Neural Networks (CNNs). During the implementation process, a deeper understanding of the workflow of image classification was gained, which involved data preprocessing, model development, training, prediction and evaluation. The comparison of the Artificial Neural Network (ANN) model and CNN model highlighted the significance of choosing the right architecture for image-related applications. CNN was found to have better classification performance because it could automatically extract image features that are important for classification and retain spatial information.

Furthermore, model optimization methods like adding more convolution layers, dropout regularization was emphasized during the enhancement phase. The results of the improvement in classification accuracy showed that the right architectural changes can have a good result on the performance. In conclusion, this tutorial was effective in reinforcing the concepts of deep learning and their application in images classification problems. The learning and skills gained from this activity are well suited for further study of AI, machine learning and computer vision applications.

Woo Cheng Shuan

This tutorial covered the basics of the entire image classification process including loading and preprocessing the dataset, training the model and testing with various training and testing datasets. The main takeaways from this tutorial is to understand the differences between ANN

and CNN, which CNN model can perform better in image classification because they are better in preserving spatial information and extracting meaningful image patterns than ANN. During training, the model can perform well on training data but poorly on new data. This is because of the training epochs or parameters that lead to overfitting. To solve this, we applied more techniques and increases training epochs to improved the original CNN model. In a nut shell, this tutorial is a very useful deep learning exercise for beginners. It applied appropriate evaluation metrics to identify the model's weaknesses and improve the reliability of predictions.